路由控制

路由控制解决的问题不是“某个数据包现在从哪条线走”，而是先决定路由信息能不能被学习、能不能被通告、能不能被重分发、能不能进入路由表。

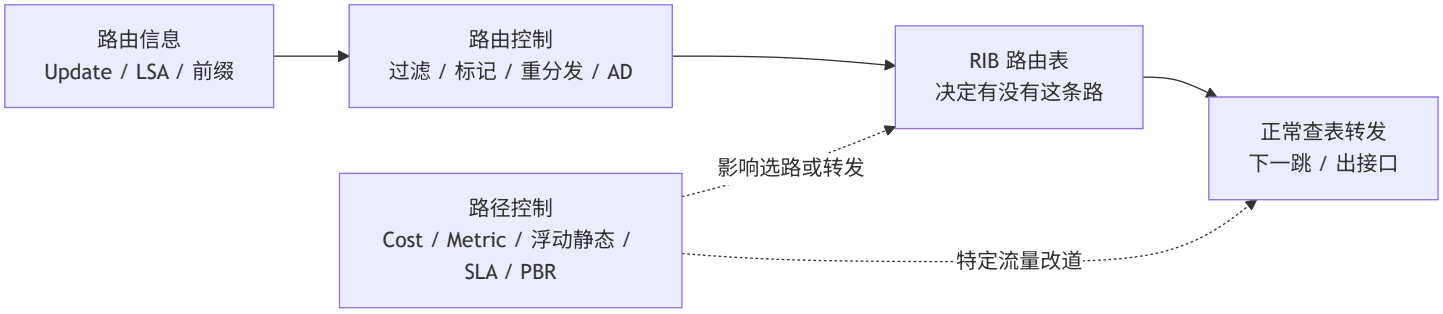
这一章按真实网络中问题出现的顺序讲：

1. 网络为什么会需要路由控制。
2. 路由器如何在多个路由来源之间做选择。
3. 用 ACL、Prefix-list、Distribute-list、Filter-list、Route-map 控制路由信息。
4. 为什么重分发最容易出问题。
5. 用拓扑和实战观察重分发导致的次优路径、反馈和环路。
6. 用 Tag、过滤和默认路由降低风险。

1. 路由控制与路径控制

路由控制和路径控制经常放在一起讲，但它们关注点不一样。

对比项	路由控制	路径控制
关注对象	路由信息	数据包实际转发路径
典型问题	这条路由能不能被学习、通告、重分发	这类流量最终走哪个出口
作用位置	控制平面	控制平面 + 数据平面
常见工具	ACL、Prefix-list、Distribute-list、Filter-list、Route-map、Tag、重分发、AD等	Cost、Metric、浮动静态、IP SLA/Track、PBR等
结果	路由表里有没有这条路由，以及这条路由是什么来源	流量下一跳、出接口或出口链路



一句话记忆：

- 路由控制先决定“有没有路”。
- 路径控制再决定“有多条路时走哪条”。

2. 为什么需要路由控制

小网络里，所有路由全部通告出去，通常也能跑起来。网络变大后，这种做法会带来明显问题。

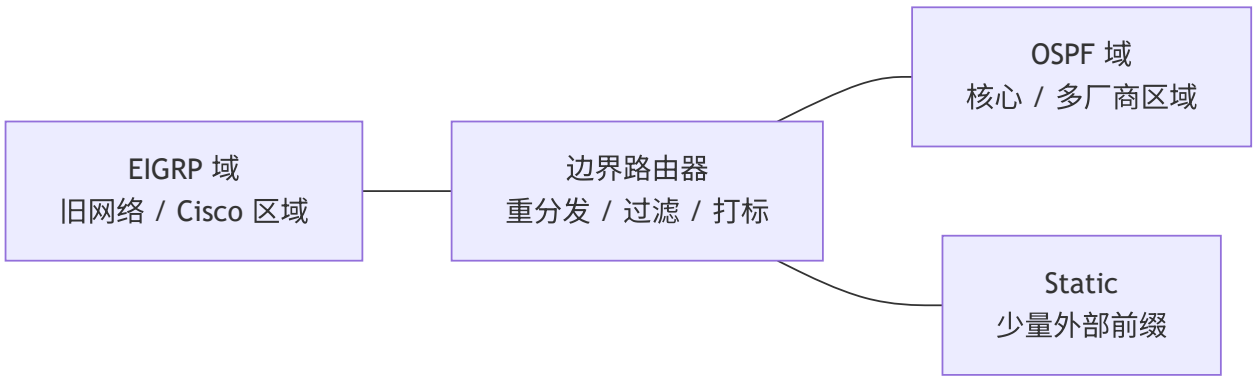
问题	表现
路由更新太多	路由器 CPU 要处理大量 Update、LSA等报文
路由表过大	内存占用增加，收敛和排障复杂度上升
协议过多	同一台边界路由器可能同时运行 EIGRP、OSPF等协议
重分发过多	路由被带到另一个协议后又被带回来，形成反馈
过滤不精确	本来不该进入核心的明细路由进入核心
Metric 不兼容	OSPF 用 Cost，EIGRP 用复合 Metric，不同协议的度量值不能直接换算

判断路由更新压力时，不只看“路由条目多不多”，还要看这些因素：

判断点	说明
更新内容长度	一次更新携带的前缀、拓扑变化或 LSA 越多，设备处理压力越大
更新频率	链路抖动、网络变化频繁时，路由协议要反复计算和通告
编址设计	地址规划越散，越难做前缀过滤和边界控制
过滤策略	过滤器写错或放错位置，可能导致过多路由被通告或必要路由被挡掉
协议进程数量	同一台边界设备运行多个路由进程时，CPU、内存和排障复杂度都会上升

复杂网络常见场景：

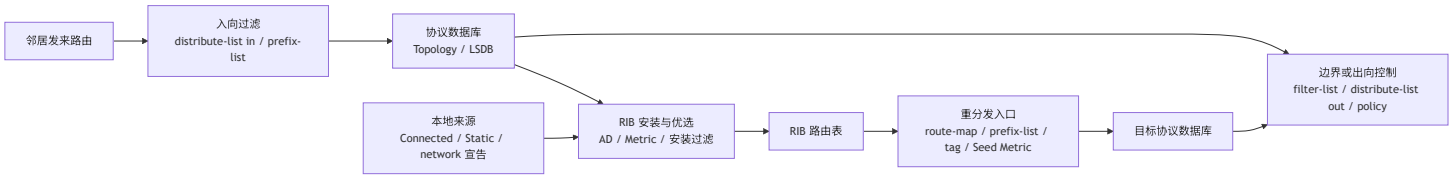
- 从旧 IGP 迁移到新 IGP，迁移期间两个协议并存。
- Cisco 区域使用 EIGRP，多厂商区域使用 OSPF。
- 分支使用 EIGRP，总部核心使用 OSPF，边界处需要做互通。
- 公司合并后，两套网络短期内不能直接统一协议。



路由控制不是为了“把命令写得更复杂”，而是为了让路由信息只在需要的范围内传播。

3. 路由控制的工具位置

路由控制可以发生在路由生命周期的不同位置。为了讲清楚，先把路由来源分成两类：一类是从邻居学来的路由，另一类是本设备产生或注入的路由。



位置	控制目标	常见工具
路由进入本设备	不让某些路由被接受或安装	`distribute-list in` 、Prefix-list
本地来源进入协议或 RIB	控制 Connected、Static、接口网段是否被使用	`network` 、静态路由、Connected 路由
协议数据库到路由表	控制本地 RIB 是否安装和优选	AD、Metric、OSPF inbound distribute-list
重分发入口	控制哪些 RIB 路由进入另一个协议	Route-map、Prefix-list、Tag、Seed Metric
边界或出向通告	减少不必要通告、不把某些路由发给邻居	Filter-list、 `distribute-list out` 、Policy

控制平面和数据平面要分清楚：

- ACL 放在接口上过滤用户流量，是数据平面。
- ACL 被 `distribute-list` 或 `route-map` 调用匹配路由，是控制平面。

3.1 工具作用速查

这些工具经常一起出现，但分工不一样。讲课时先让学生分清“匹配工具、调用位置、修改动作、防环标记”四类。

工具	主要作用	常见调用位置	记忆点
ACL	匹配地址或流量	Distribute-list、Route-map	能用，但匹配前缀长度不如 Prefix-list 清楚
Prefix-list	精确匹配路由前缀和掩码长度	Distribute-list、Filter-list、Route-map	路由前缀过滤首选
Distribute-list	控制路由接收或发送	路由协议进程	EIGRP 和 OSPF 效果不同
Filter-list	控制 OSPF 区域间 Type 3 LSA	OSPF ABR	常配合 Prefix-list 使用

工具	主要作用	常见调用位置	记忆点
Route-map	按条件匹配并执行动作	重分发、过滤	类似 if-then，permit/deny 含义看调用位置
Route Tag	给外部路由打标	重分发 Route-map	多点或双向重分发防回流
AD	比较不同路由来源可信度	RIB 安装与优选	数值越小越优先
Offset-list	增加路由 Metric	EIGRP 入向或出向	只增不减， 影响选路但不过滤路由
Seed Metric	给重分发路由设置初始 Metric	`redistribute` 或 Route-map	不同协议互相注入时必须考虑
默认路由	减少明细路由注入范围	静态路由、重分发、 协议默认注入	能解决可达性时， 不要全量互相注入
重分发	把一种来源的 RIB 路由注入另一种协议	边界路由器	风险点在过滤、Metric、AD 和 Tag

4. 管理距离 AD

当同一目的前缀来自多个路由来源时，路由器先比较管理距离 AD。AD 越小，路由来源越可信。

路由来源	默认 AD
Connected	0
Static	1
EIGRP Summary	5
eBGP	20
EIGRP Internal	90
IGRP	100
OSPF	110
IS-IS	115
RIP	120
EGP	140
ODR	160
EIGRP External	170

路由来源	默认 AD
iBGP	200
Unknown	255

这张表只用于说明不同路由来源的默认 AD。后面没讲到的协议先不用展开，先记住：同一前缀来自不同来源时，AD 小的来源优先进入 RIB。

```
D    10.10.10.0/24 [90/3072] via 10.0.13.3
O    10.10.10.0/24 [110/20]  via 10.0.12.2
```

这两条路由目的前缀一样，EIGRP Internal 的 AD 90 小于 OSPF 的 110，所以 EIGRP 路由进入路由表。

4.1 修改 AD

修改 AD 可以改变不同来源的优先级，但要非常谨慎。

```
router ospf 1
 distance 115
```

OSPF 可以按路由类型分别调整：

```
router ospf 1
 distance ospf intra-area 80 inter-area 90 external 150
```

EIGRP 内部/外部 AD：

```
router eigrp 100
 distance eigrp 90 170
```

修改 AD 的风险：

- 可能让回流路由优于原始路由。
- 可能造成次优路径。
- 可能在重分发环境中诱发环路。
- 排障时路由来源不再符合默认经验。

4.2 浮动静态路由

浮动静态路由利用较高 AD 做备份。

```
ip route 10.10.10.0 255.255.255.0 192.0.2.1 150
```

如果主路由来自 OSPF，AD 为 110，这条 AD 150 的静态路由正常情况下不会进入路由表。OSPF 路由消失后，静态路由才浮上来。

5. ACL 与 Prefix-list

ACL 和 Prefix-list 都可以被路由策略调用，但它们适合的场景不同。

对比项	ACL	Prefix-list
原始用途	数据包过滤	路由前缀过滤
匹配对象	IP 地址位	网络号 + 掩码长度
前缀长度匹配	不擅长	支持 `ge` / `le`
可读性	一般	更清楚
适合场景	精确匹配少量地址或简单网段	按前缀和掩码长度批量匹配路由

ACL 能匹配地址位，所以“匹配某个具体网段”没有问题。

```
access-list 10 permit 10.1.0.0 0.0.255.255
```

这条 ACL 可以匹配 `10.1.0.0/16` 这个地址范围。但 ACL 不擅长表达“前缀长度”这个条件。

比如下面这些需求，用 ACL 就很别扭：

需求	ACL 的问题	Prefix-list 写法
只匹配 `10.1.0.0/16` 本身	ACL 只能匹配地址范围，不直接表达“只能是 /16”	`ip prefix-list P1 permit 10.1.0.0/16`
匹配 `10.1.0.0/16` 下面所有 `/24` 明细	ACL 很难限定“必须是 /24”	`ip prefix-list P2 permit 10.1.0.0/16 ge 24 le 24`
匹配 `10.1.0.0/16` 下面 `/24` 到 `/32`	ACL 很难表达掩码长度范围	`ip prefix-list P3 permit 10.1.0.0/16 ge 24 le 32`
匹配所有路由	ACL 可以写 `permit any`，但在路由前缀语义上不够直观	`ip prefix-list ALL permit 0.0.0.0/0 le 32`

所以在路由控制中，ACL 可以用，但 Prefix-list 更适合做路由前缀过滤。

5.1 Prefix-list 语法

口诀：

```
斜杠前面定网段
ge 表示掩码长度大于等于多少
le 表示掩码长度小于等于多少
```

基本格式：

```
ip prefix-list NAME seq 10 permit 10.0.0.0/8 ge 16 le 24
```

语法拆开看：

字段	含义
`ip prefix-list`	固定命令，表示创建或修改 IPv4 前缀列表
`NAME`	Prefix-list 名字，用来被 Distribute-list、Filter-list 或 Route-map 调用
`seq 10`	序号，决定匹配顺序；不写时设备会自动生成
`permit / deny`	匹配后允许或拒绝，最终含义取决于被谁调用
`10.0.0.0/8`	基准前缀，斜杠前面定网络范围，斜杠后面定基础掩码长度
`ge 16`	允许匹配的最小掩码长度是 `/16`
`le 24`	允许匹配的最大掩码长度是 `/24`

这条命令的完整含义是：匹配属于 `10.0.0.0/8` 范围内，并且掩码长度在 `/16` 到 `/24` 之间的路由。

Prefix-list 是按 `seq` 从小到大依次匹配的。匹配到第一条后就停止，不会继续往后看。

```
ip prefix-list TEST seq 10 deny 10.1.20.0/24
ip prefix-list TEST seq 20 permit 10.1.0.0/16 ge 24 le 24
ip prefix-list TEST seq 30 permit 0.0.0.0/0 le 32
```

匹配关系可以这样理解：

层级	关系	说明
单条语句内部	与	前缀必须落在基准网段内，掩码长度也必须满足 `ge` / `le`
多条 `seq` 之间	有序匹配	从小到大检查，第一条命中就结束
多条 permit 的效果	类似或	命中任意一条 permit 就允许通过
deny 条目	例外或拦截	deny 放在前面时，可以先拦截某个更具体前缀

层级	关系	说明
列表末尾	隐式 deny	没有匹配任何条目时，默认拒绝

以上面的 `TEST` 为例：

被检查的前缀	匹配结果	原因
`10.1.20.0/24`	deny	先命中 seq 10，直接停止
`10.1.30.0/24`	permit	不命中 seq 10，命中 seq 20
`10.1.30.0/25`	permit	不命中 seq 20，因为 seq 20 只允许 `/24`；继续命中 seq 30
`192.168.1.0/24`	permit	命中 seq 30 的任意 IPv4 路由

如果想测试某个前缀会不会被 Prefix-list 匹配，可以用：

```
show ip prefix-list TEST 10.1.20.0/24 first-match
show ip prefix-list TEST 10.1.30.0/24 first-match
```

如果想看某个前缀下面有哪些更具体条目，可以用：

```
show ip prefix-list TEST 10.1.0.0/16 longer
```

常见写法：

写法	含义
`10.0.0.0/8`	只匹配 `10.0.0.0/8` 本身
`10.0.0.0/8 le 16`	匹配 `10.0.0.0/8` 到 `/16`
`10.0.0.0/8 ge 16`	匹配 `10.0.0.0/8` 下 `/16` 到 `/32`
`10.0.0.0/8 ge 16 le 24`	匹配 `/16` 到 `/24`
`0.0.0.0/0`	只匹配默认路由
`0.0.0.0/0 le 32`	匹配所有 IPv4 路由

示例：只匹配 `10.1.0.0/16` 本身。

```
ip prefix-list ONLY-10-1 permit 10.1.0.0/16
```

示例：匹配 `10.1.0.0/16` 下所有 `/24` 到 `/32` 明细。

```
ip prefix-list DETAIL-10-1 permit 10.1.0.0/16 ge 24 le 32
```

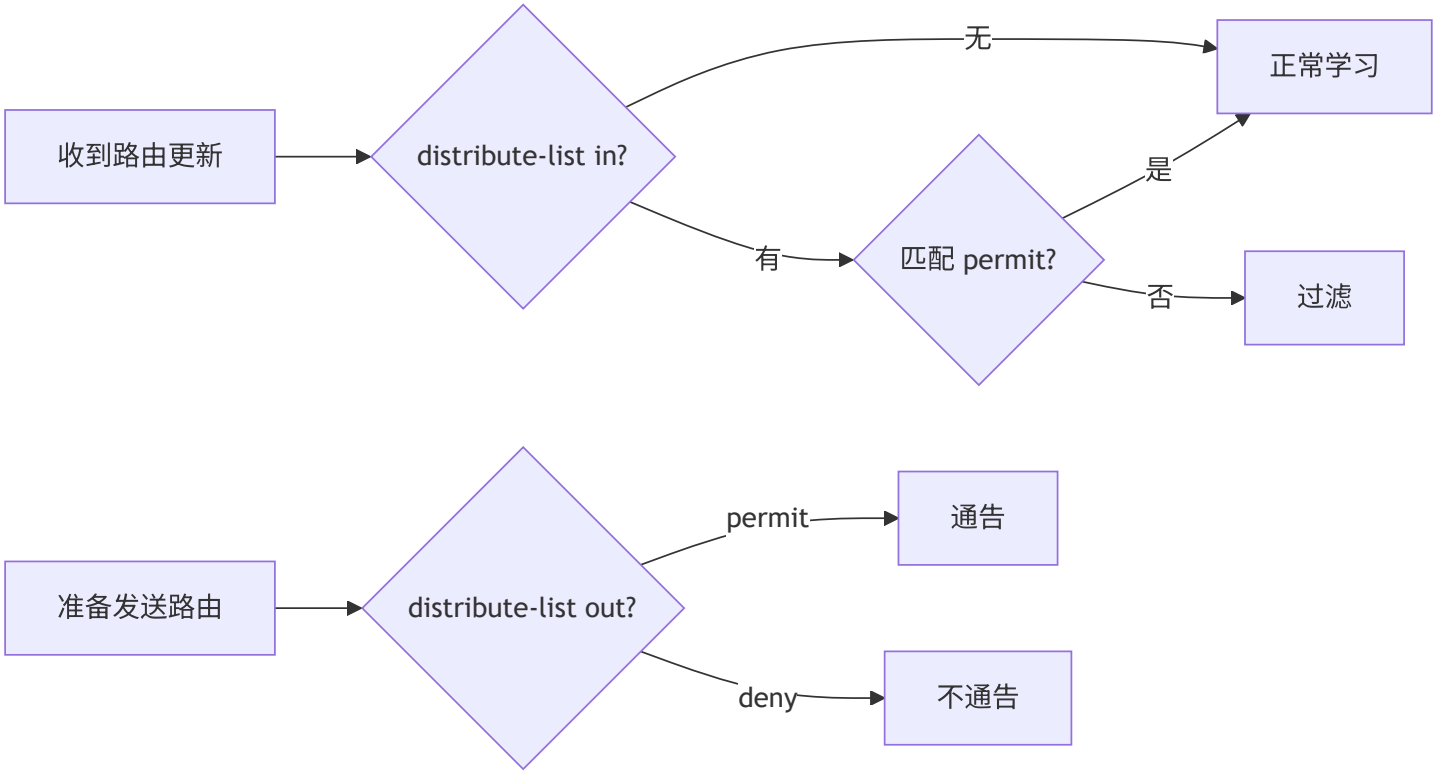
查看与清除计数器：

```
show ip prefix-list
show ip prefix-list DETAIL-10-1
show ip prefix-list detail DETAIL-10-1
show ip prefix-list DETAIL-10-1 10.1.1.0/24 first-match
show ip prefix-list DETAIL-10-1 10.1.0.0/16 longer
clear ip prefix-list DETAIL-10-1
```

6. Distribute-list

Distribute-list 是应用在路由协议进程中的路由过滤工具，用来控制哪些路由可以被接收或发送。

方向	含义
`in`	控制收到的路由是否被接受或安装
`out`	控制本设备向邻居通告哪些路由



配置前先回答三个问题：

问题	说明
过滤谁	明确要允许或拒绝的前缀，先写 Prefix-list 或 ACL

问题	说明
过滤方向	入向控制“本设备收不收”，出向控制“本设备发不发”
放在哪个接口或进程	按路由进入或离开的方向选择接口；涉及重分发时还要看来源协议

常见错误不是命令不能敲，而是过滤器放错接口、方向写反，或者调用的列表类型不对。

6.1 各协议中的差异

协议	入向过滤	出向过滤	说明
EIGRP	可以影响学习	可以影响通告	可配合 ACL、Prefix-list、Route-map
OSPF	通常只影响本地 RIB 安装	主要对重分发路由有效	同一区域 LSDB 必须一致，不能随意过滤 LSA

OSPF 的关键点：

- `distribute-list in` 不能阻止 LSA 进入 LSDB。
- 它通常只阻止 SPF 结果安装进本地路由表。
- OSPF 区域内不能像 EIGRP 那样直接按路由更新过滤拓扑信息。
- 区域间前缀过滤通常使用 `area filter-list prefix` ，位置在 ABR。

6.2 OSPF Filter-list

OSPF 的 Filter-list 常用来控制区域间路由，也就是控制 ABR 是否生成或接收某些 Type 3 LSA。它不是拿来随便过滤同一区域内部 LSA 的工具。

基本语法：

```
router ospf 1
 area AREA-ID filter-list prefix PREFIX-LIST in
 area AREA-ID filter-list prefix PREFIX-LIST out
```

方向要按“指定区域”理解：

方向	含义
`in`	控制其他区域的前缀是否进入这个区域
`out`	控制这个区域的前缀是否通告到其他区域

示例：Area 10 中有 `10.10.20.0/24` ，不希望它被 ABR 通告到其他区域。

```
ip prefix-list AREA10-OUT seq 10 deny 10.10.20.0/24
ip prefix-list AREA10-OUT seq 20 permit 0.0.0.0/0 le 32

router ospf 1
 area 10 filter-list prefix AREA10-OUT out
```

验证：

```
show ip prefix-list AREA10-OUT
show ip ospf database summary
show ip route ospf
```

记忆方式：

- `distribute-list in` 更偏向控制“本设备装不装进 RIB”。
- `area filter-list prefix` 更偏向控制“区域间有没有这条 Type 3 LSA”。
- 外部路由的控制仍然优先放在重分发入口，用 Route-map 和 Prefix-list 限定。

6.3 基础策略统一拓扑

本章中 Distribute-list、Offset-list、Route-map 这些基础工具统一使用下面这个三台路由器拓扑演示。



地址规划：

设备	接口	地址
R1	`Ethernet0/0`	`10.0.12.1/24`
R2	`Ethernet0/0`	`10.0.12.2/24`
R2	`Ethernet0/1`	`10.0.23.2/24`
R3	`Ethernet0/0`	`10.0.23.3/24`
R3	`Loopback10`	`10.3.10.3/24`
R3	`Loopback20`	`10.3.20.3/24`

基础 EIGRP 配置：

R1：

```
interface Ethernet0/0
 ip address 10.0.12.1 255.255.255.0
 no shutdown

router eigrp 100
 network 10.0.12.0 0.0.0.255
```

R2：

```
interface Ethernet0/0
 ip address 10.0.12.2 255.255.255.0
 no shutdown

interface Ethernet0/1
 ip address 10.0.23.2 255.255.255.0
 no shutdown

router eigrp 100
 network 10.0.12.0 0.0.0.255
 network 10.0.23.0 0.0.0.255
```

R3：

```
interface Ethernet0/0
 ip address 10.0.23.3 255.255.255.0
 no shutdown

interface Loopback10
 ip address 10.3.10.3 255.255.255.0

interface Loopback20
 ip address 10.3.20.3 255.255.255.0

router eigrp 100
 network 10.0.23.0 0.0.0.255
 network 10.3.10.0 0.0.0.255
 network 10.3.20.0 0.0.0.255
```

6.4 实战一：EIGRP 入向与出向路由过滤

需求：R3 发布两个业务网段，R1 只允许学习 `10.3.10.0/24`，过滤 `10.3.20.0/24`。

做法一：在 R1 做入向过滤。路由已经从 R3 发出，到达 R1 时被 R1 拦住。

```
ip prefix-list R3-IN seq 10 deny 10.3.20.0/24
ip prefix-list R3-IN seq 20 permit 0.0.0.0/0 le 32

router eigrp 100
 distribute-list prefix R3-IN in Ethernet0/0
```

做法二：在 R3 做出向过滤。R3 在发给 R2 之前就不通告 `10.3.20.0/24` 。

```
ip prefix-list R3-OUT seq 10 deny 10.3.20.0/24
ip prefix-list R3-OUT seq 20 permit 0.0.0.0/0 le 32

router eigrp 100
 distribute-list prefix R3-OUT out Ethernet0/0
```

这两种做法不要在同一次观察里同时配置。否则 R1 看不到 `10.3.20.0/24` ，但不容易判断是 R3 没发，还是 R1 收到后过滤。

验证：

```
show ip prefix-list R3-IN
show ip prefix-list R3-OUT
show running-config | section router eigrp
show ip route eigrp
show ip eigrp topology 10.3.20.0/24
```

如果过滤前已经学到了路由，改完后可以清 EIGRP 邻居或等待重新收敛后再验证：

```
clear ip eigrp neighbors
```

预期现象：

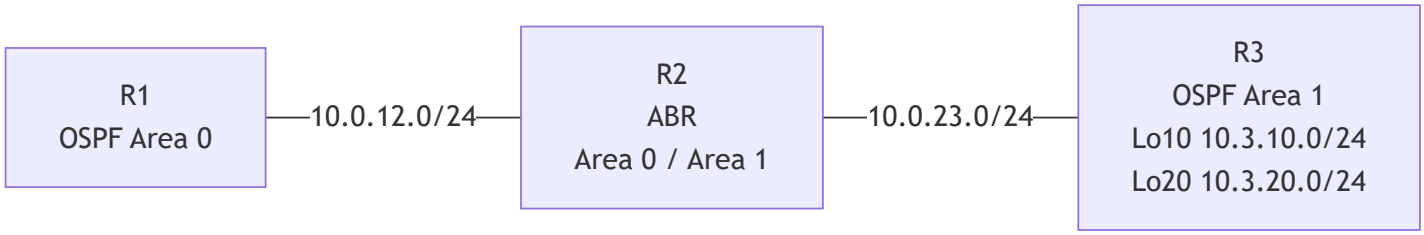
- R1 路由表中有 `10.3.10.0/24` 。
- R1 路由表中没有 `10.3.20.0/24` 。
- 做入向过滤时，R1 的 Prefix-list 命中计数能看到 `10.3.20.0/24` 被 deny。
- 做出向过滤时，R3 的 Prefix-list 命中计数能看到 `10.3.20.0/24` 被 deny。

排错重点：

- Prefix-list 末尾有隐式 deny，写了 deny 后一定要补 permit。
- 调用 Prefix-list 时必须写 `distribute-list prefix NAME` ，不能少 `prefix` 。
- Distribute-list 应用方向要对：入向过滤放在路由进入本设备的接口，出向过滤放在路由离开发送设备的接口。

6.5 实战二：OSPF distribute-list 与 Filter-list 对比

OSPF 这里要单独拎出来看。还是使用 R1-R2-R3 直线拓扑，只是把协议换成 OSPF：左边 R1-R2 属于 Area 0，右边 R2-R3 属于 Area 1，R2 是 ABR。



需求仍然是：R1 可以学习 `10.3.10.0/24`，但不要学习 `10.3.20.0/24`。

先看 `distribute-list in` 的效果。这个配置放在 R1 上：

```
ip prefix-list OSPF-IN seq 10 deny 10.3.20.0/24
ip prefix-list OSPF-IN seq 20 permit 0.0.0.0/0 le 32

router ospf 1
 distribute-list prefix OSPF-IN in
```

这个配置的效果是：R1 可以不把 `10.3.20.0/24` 安装进本地 RIB，但对应的 OSPF 信息仍然可能存在于 LSDB 中。

验证时要对比两个表：

```
R1# show ip prefix-list OSPF-IN
R1# show ip route ospf
R1# show ip ospf database summary
```

如果想控制 Area 1 的前缀是否继续被通告到 Area 0，要在 ABR R2 上用 `area filter-list prefix`。

R2：

```
ip prefix-list AREA1-TO-AREA0 seq 10 deny 10.3.20.0/24
ip prefix-list AREA1-TO-AREA0 seq 20 permit 0.0.0.0/0 le 32

router ospf 1
 area 1 filter-list prefix AREA1-TO-AREA0 out
```

这里的 `out` 要按 Area 1 理解：阻止 Area 1 里的 `10.3.20.0/24` 被 ABR 作为 Type 3 LSA 通告到其他区域。

验证：

```
R2# show ip prefix-list AREA1-TO-AREA0
R1# show ip ospf database summary
R1# show ip route ospf
```

预期现象：

```
show ip route ospf
有 10.3.10.0/24
没有 10.3.20.0/24


show ip ospf database summary
看不到 10.3.20.0/24 对应的 Summary LSA
```

记忆方式：

- OSPF `distribute-list in`：主要影响“本设备路由表装不装”。
- OSPF `area filter-list prefix`：影响“区域间 Type 3 LSA 有没有”。
- 同一区域内部 LSA 不能用这种方式随便过滤。

7. Offset-list

Offset-list 用于给收到或发出的路由增加 Metric，常见于 RIP 和 EIGRP。本轮课程只放在 EIGRP 场景中理解：它不是过滤路由，而是让匹配到的路由 Metric 变大，从而影响选路结果。OSPF 这类链路状态协议不使用 Offset-list。

本节仍然沿用 R1-R2-R3 直线拓扑。

特点：

- 只能增加 Metric，不能降低 Metric。
- 可以用 ACL 限定前缀。
- 可以指定 `in` 或 `out`。
- 可以指定接口。

需求：让 R1 学到 `10.3.10.0/24` 时，Metric 被人为增加；`10.3.20.0/24` 不受影响。

```
access-list 10 permit 10.3.10.0 0.0.0.255

router eigrp 100
 offset-list 10 out 10000 Ethernet0/0
```

这段配置放在 R2 上。含义是：R2 从 `Ethernet0/0` 发给 R1 的 EIGRP 路由中，如果匹配 `10.3.10.0/24`，就把该路由的 Metric 增加 `10000`。

验证：

```
show ip route 10.3.10.0
show ip route 10.3.20.0
show ip eigrp topology 10.3.10.0/24
```

预期现象：

- R1 仍然能学到 `10.3.10.0/24` 。
- `10.3.10.0/24` 的 EIGRP Metric 会比配置前变大。
- Offset-list 只是增加 Metric，不会把路由过滤掉。

Offset-list 更偏向路径影响，但它控制的是路由 Metric，因此也经常放在路由控制工具中一起讲。

8. Route-map

Route-map 是路由策略中最灵活的工具，可以理解成按序号执行的 if-then 逻辑。

```
route-map NAME permit 10
 match 条件
 set 动作

route-map NAME deny 20
 match 条件
```

8.1 Route-map 执行规则

- 从小序号到大序号依次匹配。
- 匹配到某个序号后，默认停止继续往下匹配。
- 没有 `match` 的 permit 条目通常表示匹配所有。
- 末尾有隐式 deny。
- `permit` 和 `deny` 的含义取决于 Route-map 被调用的位置。

多个 `match` 要分清楚关系：

写法	关系	说明
同一个 `seq` 里多个同类 `match`	或	例如多个 `match ip address` ，命中任意一个列表即可
同一个 `seq` 里多个不同类 `match`	与	例如同时写 `match ip address` 和 `match metric` ，两个条件都要满足
多个不同 `seq`	有序匹配	从小到大检查，命中一个 `seq` 后默认停止

例子一：同一个 `seq` 里多个同类 `match` 。

```
ip prefix-list PFX-A permit 10.1.0.0/16
ip prefix-list PFX-B permit 10.2.0.0/16

route-map TEST permit 10
 match ip address prefix-list PFX-A
 match ip address prefix-list PFX-B
```

这两个 `match ip address prefix-list` 属于同类条件，整体可以理解为：匹配 `PFX-A` 或 `PFX-B` 。

路由	结果
`10.1.0.0/16`	命中 seq 10
`10.2.0.0/16`	命中 seq 10
`10.3.0.0/16`	不命中 seq 10

例子二：同一个 `seq` 里多个不同类 `match` 。

```
ip prefix-list PFX-A permit 10.1.0.0/16

route-map TEST permit 20
 match ip address prefix-list PFX-A
 match metric 100
```

这两个 `match` 属于不同类条件，整体可以理解为：既要匹配 `PFX-A` ，又要 Metric 等于 `100` 。

路由	Metric	结果
`10.1.0.0/16`	`100`	命中 seq 20
`10.1.0.0/16`	`200`	不命中 seq 20
`10.2.0.0/16`	`100`	不命中 seq 20

例子三：多个不同 `seq` 。

```
ip prefix-list BLOCK permit 10.3.0.0/16
ip prefix-list CORE permit 10.0.0.0/8 le 24

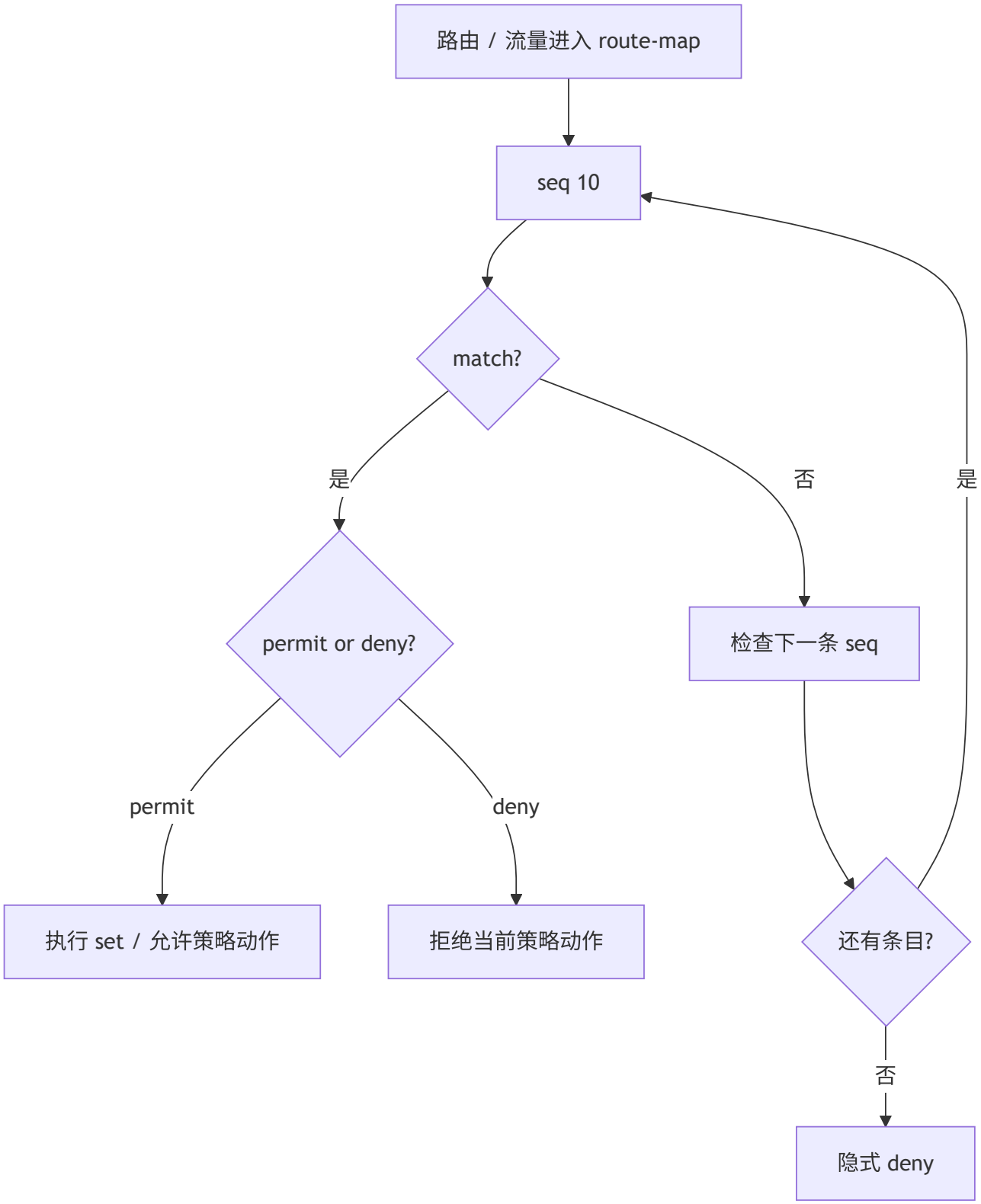
route-map TEST deny 10
 match ip address prefix-list BLOCK

route-map TEST permit 20
 match ip address prefix-list CORE

route-map TEST permit 100
```

不同 `seq` 从小到大检查，命中一个就停止。

路由	匹配过程	结果
`10.3.0.0/16`	先命中 seq 10	deny，停止，不再看 seq 20
`10.4.4.0/24`	不命中 seq 10，命中 seq 20	permit，停止
`192.168.1.0/24`	不命中 seq 10，也不命中 seq 20，命中 seq 100	permit



如果 Route-map 被用于过滤或重分发，未匹配任何 `seq` 的路由会被隐式拒绝。需要放行剩余路由时，要显式加一条空的 permit：

```
route-map TEST permit 100
```

8.2 permit / deny 在不同场景中的含义

使用位置	`permit` 匹配后	`deny` 匹配后	未匹配任何条目
重分发	允许进入目标协议，可执行 set	不重分发	不重分发
Distribute-list	允许通过	过滤	过滤

这一点非常重要：Route-map 的 `permit / deny` 不是固定含义，要看它被哪个功能调用。

8.3 常见 match 与 set

match	匹配对象	常见用途
`match ip address ACL`	ACL 匹配的地址或流量	简单过滤
`match ip address prefix-list NAME`	路由前缀	重分发、过滤
`match tag TAG`	路由 Tag	重分发防环
`match metric VALUE`	Metric	特定度量路由
`match route-type external`	路由类型	OSPF 外部路由

set	设置内容	常见用途
`set metric`	Metric	重分发种子 Metric
`set metric-type`	OSPF E1/E2	控制 OSPF 外部类型
`set tag`	Route Tag	重分发防环

9. 重分发基础

重分发是把一个路由来源注入另一个路由协议。

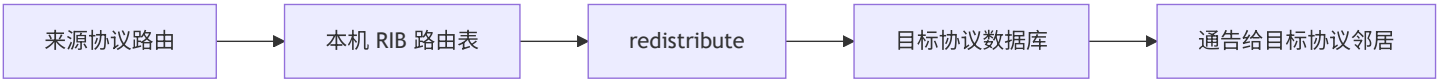
常见来源：

- Connected。
- Static。
- EIGRP。
- OSPF。

核心原则：

- 重分发是向目标协议“注入路由”，不是直接修改本地路由表。
- 被重分发的路由必须先存在于本机 RIB。
- 被重分发后，路由在目标协议中会变成目标协议的外部路由或外来路由。

- 不同协议 Metric 不兼容，必须设计种子 Metric。
- 多点或双向重分发必须考虑防环。



9.1 路由必须在 RIB 中

如果同一前缀同时来自静态路由和 OSPF，而静态路由因为 AD 更低进入了路由表，那么从 OSPF 重分发到 EIGRP 时，这条前缀不会被当作 OSPF 路由重分发。

```
S    10.10.10.0/24 [1/0]     via 10.0.13.3
O    10.10.10.0/24 [110/20]  via 10.0.12.2
```

本机 RIB 里实际安装的是静态路由。此时：

```
router eigrp 100
 redistribute ospf 1 metric 100000 100 255 1 1500
```

不会把 `10.10.10.0/24` 作为 OSPF 来源注入 EIGRP，因为这条路由在 RIB 中不是 OSPF 来源。

9.2 Seed Metric

种子 Metric 是路由进入目标协议后的初始度量。

目标协议	Metric 含义	示例
EIGRP	带宽、延迟、可靠性、负载、MTU	`metric 100000 100 255 1 1500`
OSPF	Cost	`metric 20 metric-type 1`

示例：

```
router eigrp 100
 redistribute static metric 100000 100 255 1 1500

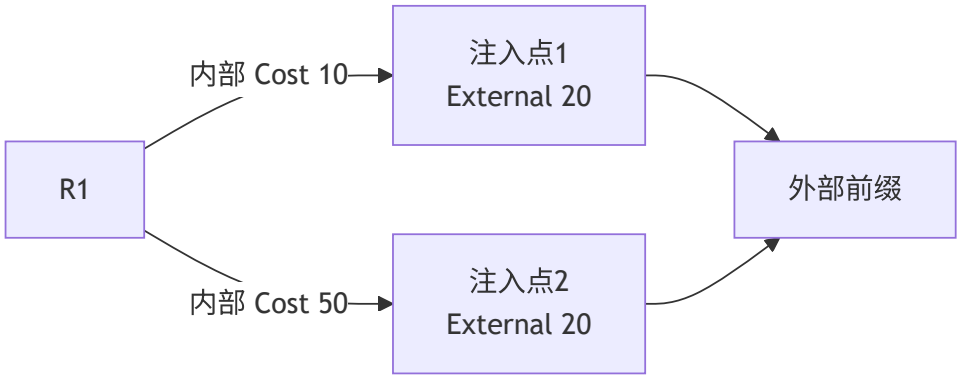

router ospf 1
 redistribute static subnets metric 20 metric-type 1
```

没有明确 Seed Metric 时，不同协议和平台行为可能不同。

9.3 OSPF E1 与 E2

OSPF 外部路由常见两种类型。

类型	计算方式	适用场景
E1	外部 Metric + 到注入点的内部 Cost	多出口注入同一前缀，想选离自己更近的出口
E2	主要比较外部 Metric	默认类型，简单但可能忽略到注入点的距离



如果是 E2，两条外部 Metric 都是 20，内部距离影响很弱。如果是 E1，总开销分别是 30 和 70，更容易选择离自己更近的注入点。

10. 重分发模型

重分发的风险主要取决于两个维度：

- 是单点还是多点。
- 是单向还是双向。

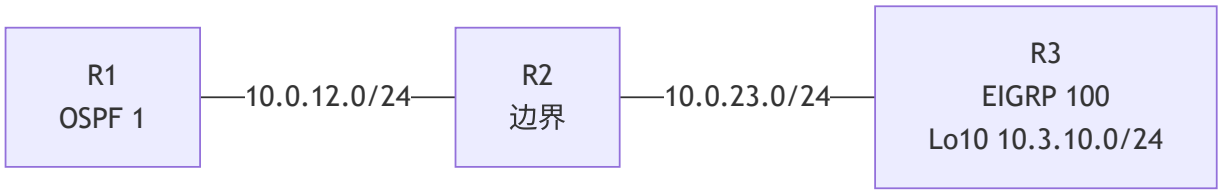
模型	风险	说明
单点单向	低	只有一个边界，只注入一个方向
单点双向	中	只有一个边界，但两个方向都注入
多点单向	中高	多个边界从 A 注入 B，可能产生次优
多点双向	高	多个边界双向注入，最容易反馈和环路

设计优先级：

1. 能单向就不要双向。
2. 能单点就不要多点。
3. 能默认路由解决就不要全量互相重分发。
4. 必须多点双向时，使用 Prefix-list、Route-map、Tag、Metric 和 AD。

11. 单点单向重分发的推荐设计

这是风险最低、最容易控制的模型。继续使用第一张 R1-R2-R3 直线拓扑：R1 属于 OSPF，R3 属于 EIGRP，R2 是唯一边界。



推荐设计：

- R2 只把 R3 的必要业务网段注入 OSPF。
- R2 不把 OSPF 明细全量注回 EIGRP。
- R2 只给 EIGRP 下发默认路由。

R2：EIGRP 注入 OSPF。

```
ip prefix-list EIGRP-BRANCH permit 10.3.10.0/24

route-map EIGRP-TO-OSPF permit 10
 match ip address prefix-list EIGRP-BRANCH
 set metric 20
 set metric-type type-1
 set tag 100

router ospf 1
 redistribute eigrp 100 subnets route-map EIGRP-TO-OSPF
```

R2：给 EIGRP 下发默认路由。

```
ip route 0.0.0.0 0.0.0.0 Null0

route-map DEFAULT-ONLY permit 10
 match ip address prefix-list DEFAULT
 set metric 100000 100 255 1 1500

ip prefix-list DEFAULT permit 0.0.0.0/0

router eigrp 100
 redistribute static route-map DEFAULT-ONLY
```

预期：

- OSPF 域能看到 `10.3.10.0/24` 外部路由。
- EIGRP 域只拿到默认路由，不学习 OSPF 全量明细。
- 没有第二个重分发点，反馈风险低。

12. 重分发导致的问题

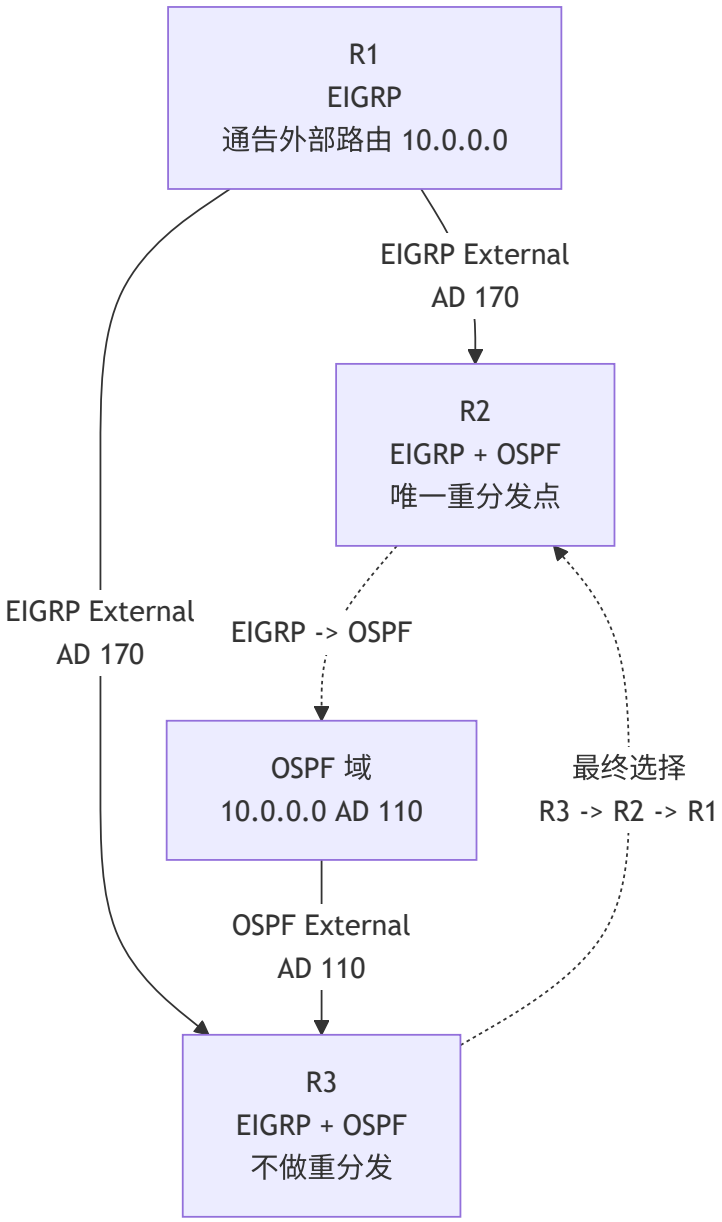
重分发的问题不是“命令写错才会出现”。很多时候， `redistribute` 命令本身能敲成功，邻居关系也正常，但路由选择已经变差。

分析这类问题时抓住三个点：

1. 同一条前缀是否被两个协议同时看见。
2. 不同协议之间先比较 AD，再比较各自协议内部 Metric。
3. 路由被重分发后，原协议的 Metric 不能原样带到目标协议。

12.1 单点单向重分发可能导致次优路径

单点单向重分发并不一定有环路，但仍然可能产生次优路径。



过程：

1. R1 运行 EIGRP，并把 `10.0.0.0` 这条外部路由通告给 R2 和 R3。
2. R2 和 R3 都同时运行 EIGRP 和 OSPF。
3. 只有 R2 把 EIGRP 路由重分发进 OSPF。
4. R3 既能从 R1 收到 EIGRP External 路由，也能从 R2 收到 OSPF External 路由。

R3 上会看到两种来源：

```
D EX 10.0.0.0/8 [170/...] via R1
O E2 10.0.0.0/8 [110/20]  via R2
```

OSPF 的 AD 是 110，EIGRP External 的 AD 是 170。R3 会选择 OSPF 路由，于是报文不再直接发给 R1，而是走：

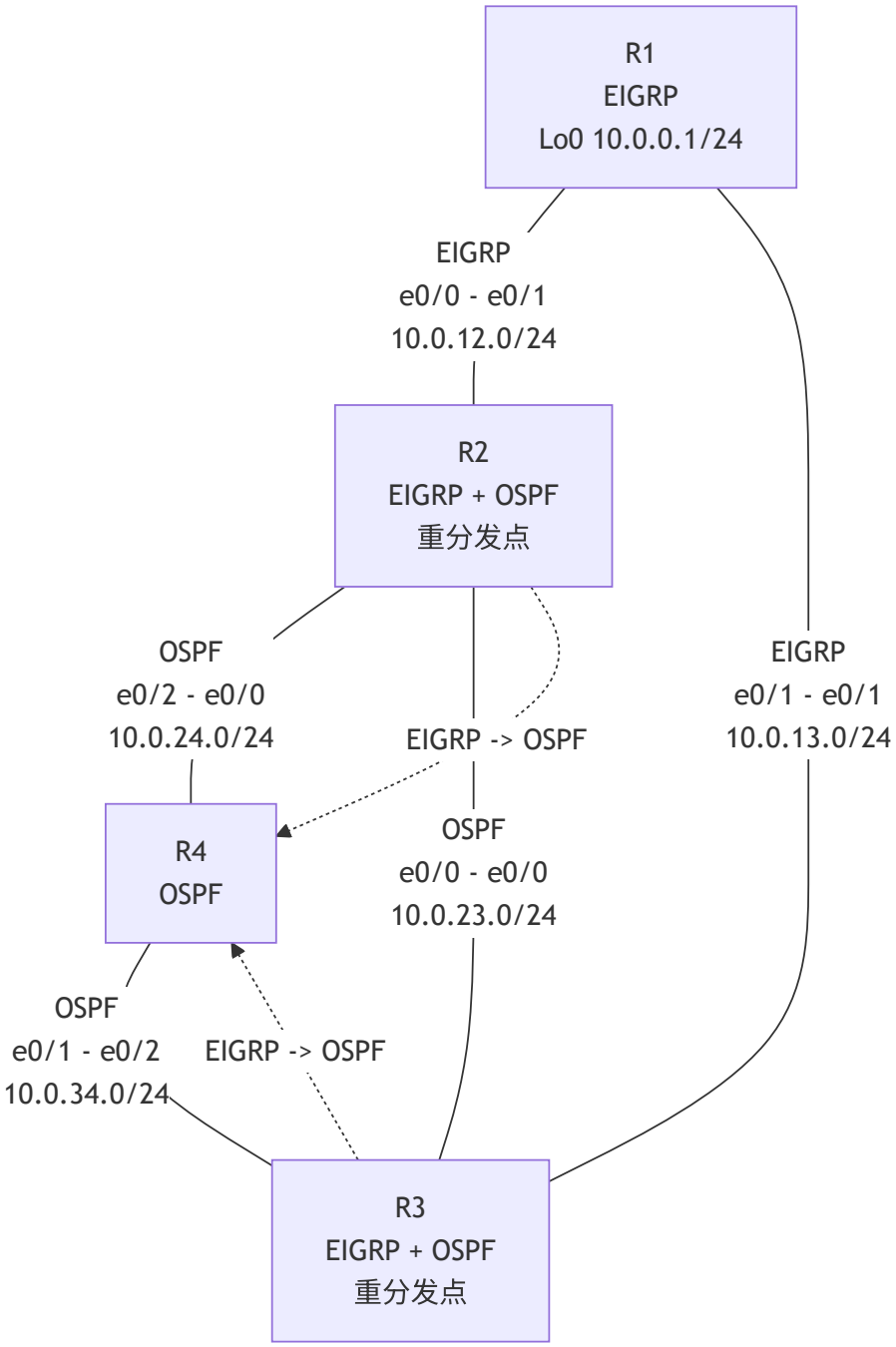
```
R3 -> R2 -> R1
```

这就是单点单向重分发也可能导致的问题：没有形成环路，但形成了次优路由。

12.2 多点单向重分发可能导致环路

多点单向重分发比单点单向更危险。方向虽然仍然只有 EIGRP 到 OSPF，但因为有两个重分发点，同一条外部路由会在两个边界之间互相影响。

本实验用下面这个拓扑演示故障效果：下面是 EIGRP，上面是 OSPF；R2、R3 同时运行 EIGRP 和 OSPF，并且都把 EIGRP 重分发进 OSPF。



地址规划：

设备	接口	地址	协议
R1	`Loopback0`	`10.0.0.1/24`	Connected，重分发进 EIGRP
R1	`Ethernet0/0`	`10.0.12.1/24`	EIGRP
R1	`Ethernet0/1`	`10.0.13.1/24`	EIGRP
R2	`Ethernet0/1`	`10.0.12.2/24`	EIGRP
R2	`Ethernet0/0`	`10.0.23.2/24`	OSPF
R2	`Ethernet0/2`	`10.0.24.2/24`	OSPF

设备	接口	地址	协议
R3	Ethernet0/1	10.0.13.3/24	EIGRP
R3	Ethernet0/0	10.0.23.3/24	OSPF
R3	Ethernet0/2	10.0.34.3/24	OSPF
R4	Ethernet0/0	10.0.24.4/24	OSPF
R4	Ethernet0/1	10.0.34.4/24	OSPF

故障配置如下。

R1：把 10.0.0.0/24 作为外部路由注入 EIGRP。

```
interface Loopback0
 ip address 10.0.0.1 255.255.255.0

interface Ethernet0/0
 ip address 10.0.12.1 255.255.255.0
 no shutdown

interface Ethernet0/1
 ip address 10.0.13.1 255.255.255.0
 no shutdown

ip prefix-list R1-LO0 seq 10 permit 10.0.0.0/24

route-map CONNECTED-TO-EIGRP permit 10
 match ip address prefix-list R1-LO0

router eigrp 100
 network 10.0.12.0 0.0.0.255
 network 10.0.13.0 0.0.0.255
 redistribute connected metric 100000 100 255 1 1500 route-map CONNECTED-TO-EIGRP
```

R2：同时运行 EIGRP 和 OSPF，并把 EIGRP 注入 OSPF。

```
interface Ethernet0/1
 ip address 10.0.12.2 255.255.255.0
 no shutdown

interface Ethernet0/0
 ip address 10.0.23.2 255.255.255.0
 no shutdown

interface Ethernet0/2
 ip address 10.0.24.2 255.255.255.0
 no shutdown

router eigrp 100
 network 10.0.12.0 0.0.0.255

router ospf 1
 router-id 2.2.2.2
 network 10.0.23.0 0.0.0.255 area 0
 network 10.0.24.0 0.0.0.255 area 0
 redistribute eigrp 100 subnets metric 20 metric-type 2
```

R3：也把 EIGRP 注入 OSPF。

```
interface Ethernet0/1
 ip address 10.0.13.3 255.255.255.0
 no shutdown

interface Ethernet0/0
 ip address 10.0.23.3 255.255.255.0
 no shutdown

interface Ethernet0/2
 ip address 10.0.34.3 255.255.255.0
 no shutdown

router eigrp 100
 network 10.0.13.0 0.0.0.255

router ospf 1
 router-id 3.3.3.3
 network 10.0.23.0 0.0.0.255 area 0
 network 10.0.34.0 0.0.0.255 area 0
 redistribute eigrp 100 subnets metric 20 metric-type 2
```

R4：只运行 OSPF。

```
interface Ethernet0/0
 ip address 10.0.24.4 255.255.255.0
 no shutdown

interface Ethernet0/1
 ip address 10.0.34.4 255.255.255.0
 no shutdown

router ospf 1
 router-id 4.4.4.4
 network 10.0.24.0 0.0.0.255 area 0
 network 10.0.34.0 0.0.0.255 area 0
```

故障过程：

1. R1 把 EIGRP 路由，包括 EIGRP External 路由，通告给 R2 和 R3。
2. R2 和 R3 都运行 EIGRP 和 OSPF。
3. R2、R3 都把 EIGRP 路由重分发进 OSPF。
4. R2 除了从 R1 收到 EIGRP External，还会从 OSPF 收到 R3 注入的同一前缀。
5. R3 除了从 R1 收到 EIGRP External，也会从 OSPF 收到 R2 注入的同一前缀。

由于 OSPF AD 110 小于 EIGRP External AD 170，R2 和 R3 都可能选择 OSPF 路由。

结果：

- 最好的情况：R2 或 R3 绕一圈再去 R1，形成次优路径。
- 更糟的情况：R2 认为去 R1 要走 R3，R3 又认为去 R1 要走 R2，形成路由环路。

```
R2 -> R3 -> R2 -> R3 ...
```

验证故障：

```
R3# show ip route 10.0.0.0
R3# ping 10.0.0.1

R4# show ip route 10.0.0.0
R4# ping 10.0.0.1
```

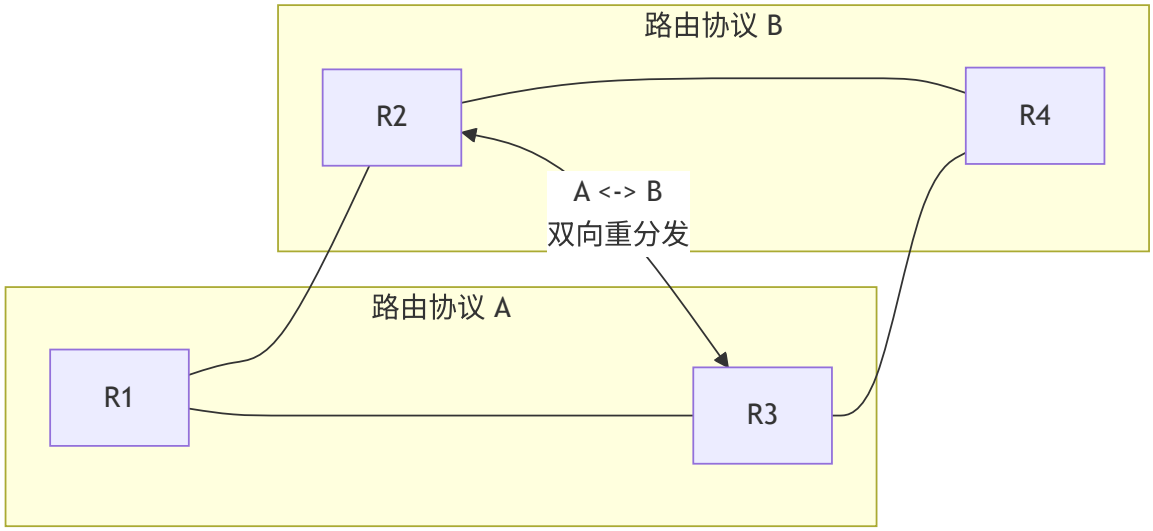
故障现象：

- R3 去 `10.0.0.1` 可能选择 OSPF 外部路由，而不是直接走 EIGRP 到 R1。
- R4 能学到 `10.0.0.0/24`，但实际转发可能被 R2/R3 的错误选择带进环路。
- R3 ping `10.0.0.1` 不通，R4 ping `10.0.0.1` 也不通。

这个实验不是推荐配置，而是专门用来演示：多点单向重分发也可能把网络做坏。解决时要用过滤、Tag、调整 AD，或者从设计上避免多个边界同时注入同一批外部路由。

12.3 多点双向重分发可能导致路由反馈

多点双向重分发是风险最高的模型。它的问题不只是“路由多”，而是路由可能从原协议出去后，又从另一个边界回到原协议。



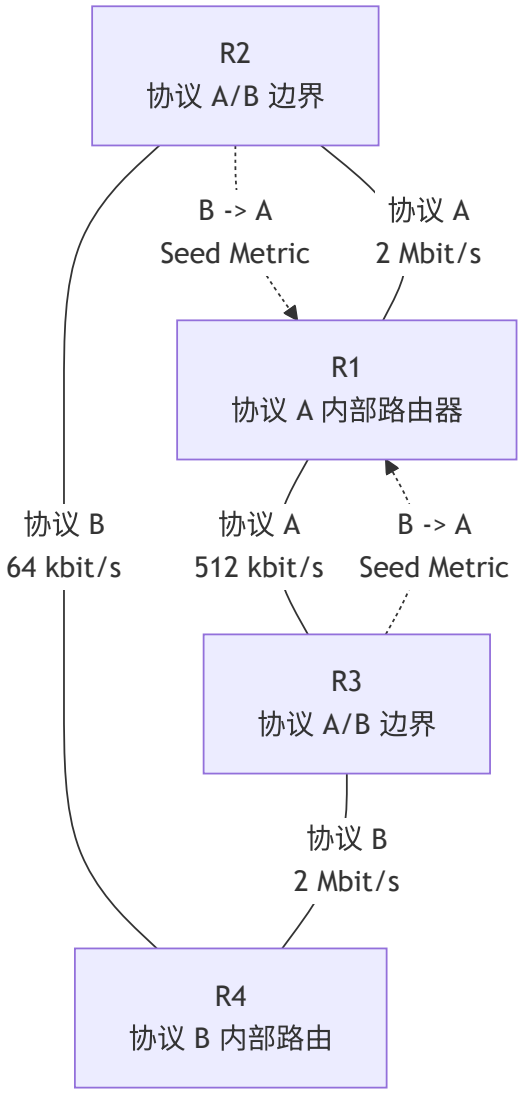
问题链条：

1. 协议 A 的路由在 R2 处被重分发进协议 B。
2. 这条路由在协议 B 中传播到 R3。
3. R3 又把这条路由重分发回协议 A。
4. 对协议 A 来说，这条路由已经从外面绕了一圈回来。

这叫路由反馈。反馈本身不一定马上形成环路，但它会制造错误候选路由。一旦 AD 或 Metric 让反馈路由优于原始路由，就可能出现次优路径或环路。

12.4 多点双向重分发可能导致 Metric 丢失

不同协议的 Metric 无法直接继承。路由从协议 B 重分发到协议 A 时，协议 B 内部真实的链路代价会丢失，只能在协议 A 中重新给一个 Seed Metric。



从协议 B 的角度看，R3 到 R4 是 2 Mbit/s，明显优于 R2 到 R4 的 64 kbit/s。但这条信息被重分发进协议 A 后，协议 A 未必还能知道协议 B 内部哪条链路更好。

于是 R1 可能只根据协议 A 内部 Metric 做决定，认为到 R2 的 2 Mbit/s 优于到 R3 的 512 kbit/s，最终选择：

```
R1 -> R2 -> R4
```

但进入协议 B 以后，R2 到 R4 实际只有 64 kbit/s。这就是 Metric 丢失导致的次优路由。

12.5 解决思路

重分发问题的解决顺序：

1. 能单点就不要多点。
2. 能单向就不要双向。
3. 能用默认路由解决，就不要互相注入所有明细。
4. 必须多点或双向时，用 Prefix-list 精确限定前缀。
5. 必须多点或双向时，用 Route Tag 阻止回流。
6. 每个重分发方向都显式配置 Seed Metric。

7. 检查 AD，避免回流路由优于原始路由。

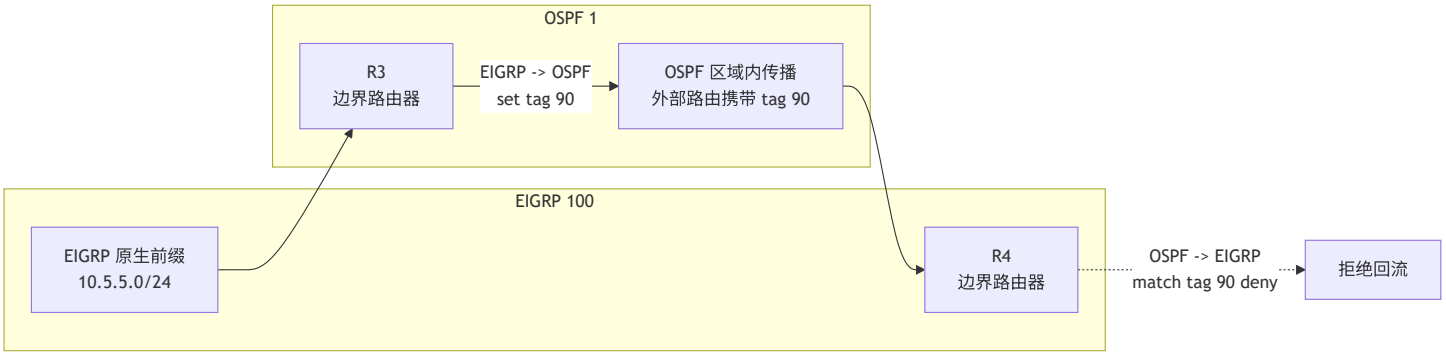
13. Route Tag 防环

Route Tag 是附加在外部路由上的标记，常用于多点或双向重分发防环。

它本身不决定数据包怎么转发，而是给路由策略留下一个“来源标记”。后续这条路由如果又走到另一个边界路由器，Route-map 就可以通过 `match tag` 判断它是不是已经被重分发过，从而阻止它回到原来的路由域。

基本思路：

1. Tag 表示“这条路由原来从哪里来”，不是表示“这条路由要去哪里”。
2. EIGRP 原生路由进入 OSPF 时打 Tag `90`。
3. OSPF 原生路由进入 EIGRP 时打 Tag `110`。
4. 任何地方做 OSPF -> EIGRP 时，拒绝 Tag `90`，防止 EIGRP 路由回到 EIGRP。
5. 任何地方做 EIGRP -> OSPF 时，拒绝 Tag `110`，防止 OSPF 路由回到 OSPF。



上图的关键点是：`10.5.5.0/24` 原来就是 EIGRP 的路由。它从 R3 进入 OSPF 后带上 Tag `90`，当它又从 OSPF 到达 R4，准备重新进入 EIGRP 时，R4 通过 `match tag 90` 识别出“这是 EIGRP 出去又绕回来的路由”，所以拒绝重分发。

反方向同理：



可以用一句话记忆：

> 谁出去，谁打来源 `Tag`；
> 谁回来，谁被来源 `Tag` 拦住。

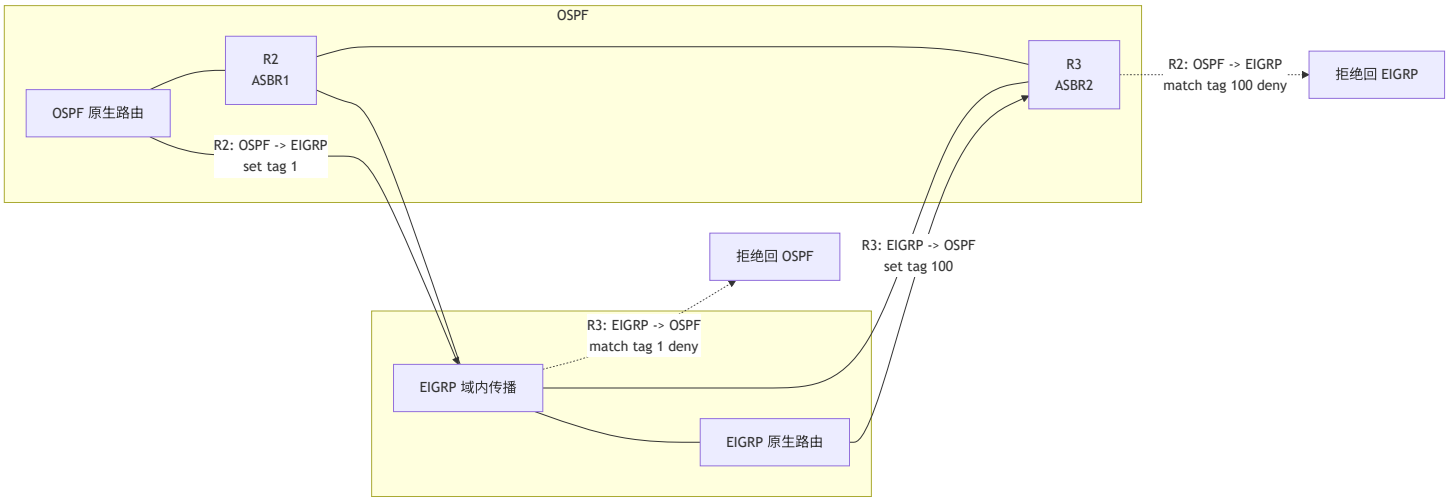
Tag 设计表：

来源路由域	注入目标协议时	回流时在哪个方向拦截	目的
EIGRP 原生路由	EIGRP ->` `OSPF： `set tag 90`	OSPF -> EIGRP： `match tag 90` 后 `deny`	不让 EIGRP 路由绕一圈再回 EIGRP
OSPF 原生路由	OSPF ->` `EIGRP： `set tag 110`	EIGRP ->` `OSPF： `match tag 110` 后 `deny`	不让 OSPF 路由绕一圈再回 OSPF

也可以不用 `90` 、 `110` ，Tag 值本身只是管理员自己定义的标记。例如：

重分发方向	设置 Tag	Tag 的含义
OSPF -> EIGRP	`set tag 1`	这条路由原来来自 OSPF
EIGRP -> OSPF	`set tag 100`	这条路由原来来自 EIGRP

在单点双向重分发时，这个设计不一定马上看出效果，因为进出都在同一台边界路由器上。到了多点重分发，Tag 的意义就很明显了。



所以判断方向时不要背“进 OSPF 打几、进 EIGRP 打几”，而是反过来想：

当前要做的重分发	必须先拒绝什么 Tag	原因
OSPF -> EIGRP	拒绝 EIGRP 来源 Tag，例如 `100`	这类路由原来就是 EIGRP 的，不能再回 EIGRP
EIGRP -> OSPF	拒绝 OSPF 来源 Tag，例如 `1`	这类路由原来就是 OSPF 的，不能再回 OSPF

配置骨架如下，下面的 Prefix-list 名称只表示“允许被重分发的前缀集合”，实际实验中要按需求提前定义。

EIGRP 注入 OSPF 时，要先拒绝 Tag `110` ，因为 Tag `110` 代表“这条路由本来从 OSPF 来”。如果不拒绝，它就会被重新注入 OSPF：

```
route-map EIGRP-TO-OSPF deny 10
 match tag 110

route-map EIGRP-TO-OSPF permit 20
 match ip address prefix-list EIGRP-TO-OSPF-PFX
 set metric 20
 set metric-type type-1
 set tag 90

router ospf 1
 redistribute eigrp 100 subnets route-map EIGRP-TO-OSPF
```

OSPF 注入 EIGRP 时，要先拒绝 Tag `90`，因为 Tag `90` 代表“这条路由本来从 EIGRP 来”。如果不拒绝，它就会被重新注入 EIGRP：

```
route-map OSPF-TO-EIGRP deny 10
 match tag 90

route-map OSPF-TO-EIGRP permit 20
 match ip address prefix-list OSPF-TO-EIGRP-PFX
 set metric 100000 100 255 1 1500
 set tag 110

router eigrp 100
 redistribute ospf 1 route-map OSPF-TO-EIGRP
```

设计 Route Tag 时按这个顺序检查：

1. 先确定每个“来源路由域”：例如 OSPF 1、EIGRP 100。
2. 每个来源路由域分配一个独立 Tag，不要只按协议名字粗略分配。
3. 在路由离开来源域、进入目标协议时 `set tag`。
4. 在路由准备回到来源域之前，用 `deny + match tag` 拦住。
5. `deny + match tag` 必须写在前面的 sequence，先拦回流，再 permit 正常路由。
6. 后面的 permit 条目仍然建议配合 Prefix-list，只允许必要前缀被重分发。

常见错误：

错误做法	问题
只 `set tag`，但没有 `match tag deny`	Tag 只是被打上了，不能自动防环
`permit` 写在前面，`deny match tag` 写在后面	Route-map 首次匹配就停止，后面的 deny 不会生效
所有方向都用同一个 Tag	无法判断路由到底从哪个路由域来
只按协议分 Tag，不按进程或路由域分 Tag	多个 OSPF 或多个 EIGRP 同时存在时容易误判

错误做法	问题
不限制前缀，只靠 Tag	可以防回流，但不能控制重分发范围

验证时重点看三个地方：

```
show route-map
show ip ospf database external
show ip eigrp topology
```

- `show route-map` 看 `deny + match tag` 的命中次数。
- `show ip ospf database external` 看 OSPF 外部 LSA 是否携带预期 Tag。
- `show ip eigrp topology` 看 EIGRP 外部路由是否携带预期 Tag。

注意：Tag 主要用于外部路由，内部路由是否携带和保留 Tag 与协议、平台和重分发方向有关。实验时一定要结合设备输出确认实际行为。

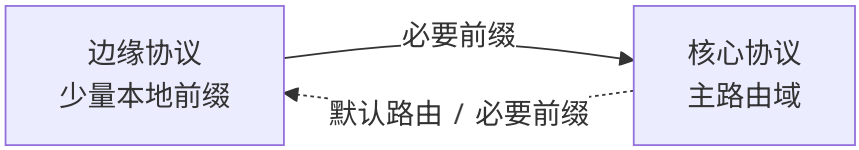
14. 核心协议与边缘协议

复杂网络不要一上来就全量双向重分发。更稳的做法是先定义核心协议和边缘协议。

类型	含义
核心协议	网络主干中承载主要路由的协议
边缘协议	旧协议、过渡协议或分支协议

常见设计：

- 边缘协议的必要前缀注入核心协议。
- 核心协议向边缘协议下发默认路由或必要前缀。
- 避免核心和边缘全量互相重分发。



这种模型能大幅减少反馈、环路和排障复杂度。

15. 默认路由

默认路由经常比全量重分发更稳定。边缘区域只需要知道“其他目的都去核心”时，不必把核心里的所有明细都注入进去。

方法	作用
默认路由	让边缘协议知道“其他都去核心”
Prefix-list	只允许必要前缀进入目标协议
Route-map	匹配、过滤、设置 Metric 和 Tag

OSPF 下发默认：

```
ip route 0.0.0.0 0.0.0.0 203.0.113.1

router ospf 1
 default-information originate
```

EIGRP 通过静态默认重分发：

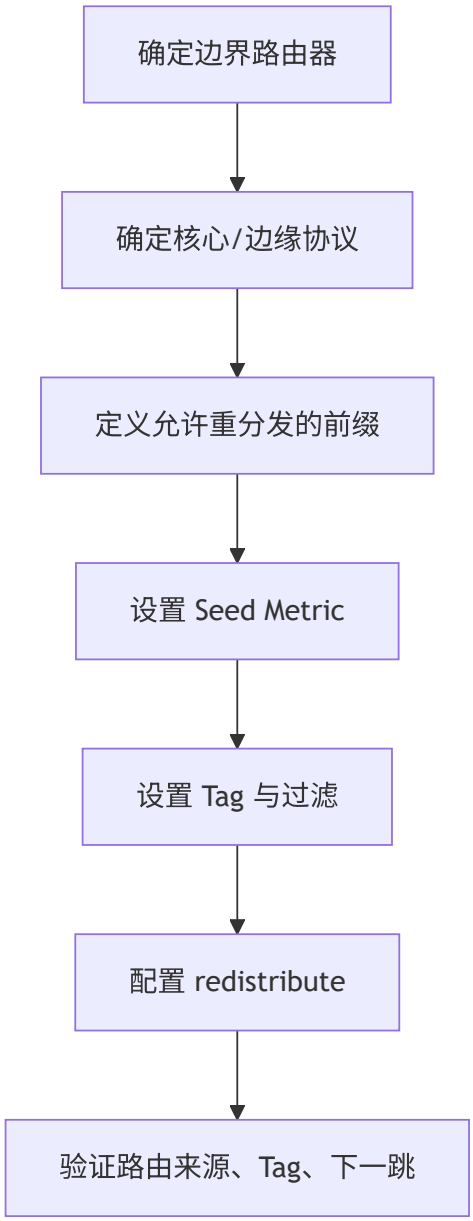
```
ip route 0.0.0.0 0.0.0.0 Null0

router eigrp 100
 redistribute static metric 100000 100 255 1 1500
```

16. 重分发实施流程

实施前先回答这些问题：

1. 哪些设备是边界路由器。
2. 哪个协议是核心协议。
3. 哪个协议是边缘协议。
4. 源协议和目标协议是否属于同一协议栈，例如 IPv4 路由不能直接重分发进 IPv6 路由进程。
5. 哪些前缀必须进入核心。
6. 哪些前缀可以用默认路由代替。
7. 目标协议需要什么 Seed Metric。
8. 是否存在多个重分发点。
9. 是否需要 Route Tag 防环。
10. 是否需要调整 AD 或 Metric。
11. 如何验证和回退。



通用命令结构：

```
redistribute protocol [process-id] [metric metric-value] [metric-type type] [subnets] [route-map name]
```

常见参数：

参数	含义
`protocol`	被重分发的来源协议
`process-id`	来源协议进程号或协议编号
`metric`	进入目标协议的 Seed Metric
`metric-type`	OSPF 外部类型
`subnets`	OSPF 中包含子网路由

参数	含义
`route-map`	调用策略控制匹配、过滤、设置属性

17. 综合实战：OSPF 与 EIGRP 双向重分发

这个实战使用六台路由器。左侧是 OSPF，右侧是 EIGRP；R3、R4 是两台 ASBR，在两个边界点同时做双向重分发。

实验目标：

- OSPF 使用进程 `1`，链路地址使用 `10.1.x.x`。
- EIGRP 使用 AS `100`，链路地址使用 `10.100.x.x`。
- R3、R4 同时运行 OSPF 和 EIGRP，并在两边做双向重分发。
- OSPF -> EIGRP 的路由打 Tag `1`。
- EIGRP -> OSPF 的路由打 Tag `100`。
- 通过 Prefix-list + Route-map + Tag 防止路由从一个协议出去以后又绕回原协议。

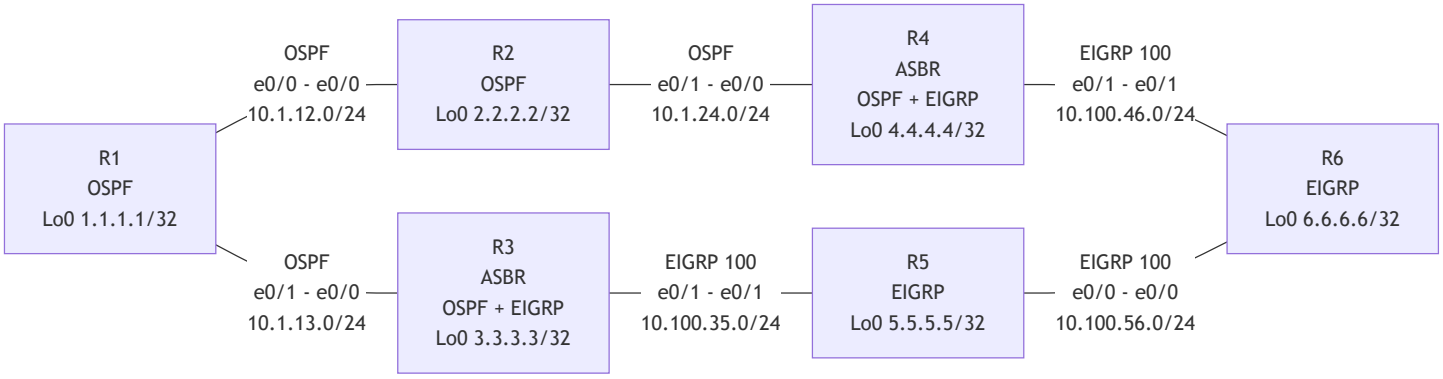
本实验所有设备都使用 `Loopback0`：

设备	Loopback0
R1	`1.1.1.1/32`
R2	`2.2.2.2/32`
R3	`3.3.3.3/32`
R4	`4.4.4.4/32`
R5	`5.5.5.5/32`
R6	`6.6.6.6/32`

本实验约定：

- R1/R2/R3/R4 的 Loopback0 属于 OSPF 原生前缀。
- R5/R6 的 Loopback0 属于 EIGRP 原生前缀。
- R3、R4 本机的 Loopback0 虽然是 Connected，但通过 OSPF 宣告进入 OSPF 域；在重分发时，通常由另一台 ASBR 学到后再注入 EIGRP。

17.1 拓扑



地址规划：

链路/网段	地址
R1-R2	`10.1.12.0/24`
R1-R3	`10.1.13.0/24`
R2-R4	`10.1.24.0/24`
R3-R5	`10.100.35.0/24`
R4-R6	`10.100.46.0/24`
R5-R6	`10.100.56.0/24`
OSPF 原生前缀	`1.1.1.1/32` 、 `2.2.2.2/32` 、 `3.3.3.3/32` 、 `4.4.4.4/32`
EIGRP 原生前缀	`5.5.5.5/32` 、 `6.6.6.6/32`

17.2 基础协议配置

R1：

```
interface Ethernet0/0
 ip address 10.1.12.1 255.255.255.0
 no shutdown

interface Ethernet0/1
 ip address 10.1.13.1 255.255.255.0
 no shutdown

interface Loopback0
 ip address 1.1.1.1 255.255.255.255

router ospf 1
 router-id 1.1.1.1
 network 1.1.1.1 0.0.0.0 area 1
 network 10.1.12.0 0.0.0.255 area 1
 network 10.1.13.0 0.0.0.255 area 1
```

R2：

```
interface Ethernet0/0
 ip address 10.1.12.2 255.255.255.0
 no shutdown

interface Ethernet0/1
 ip address 10.1.24.2 255.255.255.0
 no shutdown

interface Loopback0
 ip address 2.2.2.2 255.255.255.255

router ospf 1
 router-id 2.2.2.2
 network 2.2.2.2 0.0.0.0 area 1
 network 10.1.12.0 0.0.0.255 area 1
 network 10.1.24.0 0.0.0.255 area 1
```

R3：

```
interface Ethernet0/0
 ip address 10.1.13.3 255.255.255.0
 no shutdown

interface Ethernet0/1
 ip address 10.100.35.3 255.255.255.0
 no shutdown

interface Loopback0
 ip address 3.3.3.3 255.255.255.255

router ospf 1
 router-id 3.3.3.3
 network 3.3.3.3 0.0.0.0 area 1
 network 10.1.13.0 0.0.0.255 area 1

router eigrp 100
 no auto-summary
 network 10.100.35.0 0.0.0.255
```

R4：

```
interface Ethernet0/0
 ip address 10.1.24.4 255.255.255.0
 no shutdown

interface Ethernet0/1
 ip address 10.100.46.4 255.255.255.0
 no shutdown

interface Loopback0
 ip address 4.4.4.4 255.255.255.255

router ospf 1
 router-id 4.4.4.4
 network 4.4.4.4 0.0.0.0 area 1
 network 10.1.24.0 0.0.0.255 area 1

router eigrp 100
 no auto-summary
 network 10.100.46.0 0.0.0.255
```

R5：

```
interface Ethernet0/0
 ip address 10.100.56.5 255.255.255.0
 no shutdown

interface Ethernet0/1
 ip address 10.100.35.5 255.255.255.0
 no shutdown

interface Loopback0
 ip address 5.5.5.5 255.255.255.255

router eigrp 100
 no auto-summary
 network 5.5.5.5 0.0.0.0
 network 10.100.35.0 0.0.0.255
 network 10.100.56.0 0.0.0.255
```

R6：

```
interface Ethernet0/0
 ip address 10.100.56.6 255.255.255.0
 no shutdown

interface Ethernet0/1
 ip address 10.100.46.6 255.255.255.0
 no shutdown

interface Loopback0
 ip address 6.6.6.6 255.255.255.255

router eigrp 100
 no auto-summary
 network 6.6.6.6 0.0.0.0
 network 10.100.46.0 0.0.0.255
 network 10.100.56.0 0.0.0.255
```

17.3 重分发前检查

在 R3、R4 上先确认两边邻居和基础路由都正常。

R3：

```
show ip ospf neighbor
show ip eigrp neighbors
show ip route ospf
show ip route eigrp
```

R4：

```
show ip ospf neighbor
show ip eigrp neighbors
show ip route ospf
show ip route eigrp
```

预期：

- R3、R4 能通过 OSPF 学到 OSPF 域内的 Loopback0。
- R3、R4 能通过 EIGRP 学到 `5.5.5.5/32` 、 `6.6.6.6/32` 。
- 此时 OSPF 侧还不能到达 EIGRP 侧 Loopback0，EIGRP 侧也还不能到达 OSPF 侧 Loopback0。

17.4 Tag 设计

这里的 Tag 直接按来源路由域设计：

当前重分发方向	先拒绝什么	放行后打什么 Tag	含义
OSPF -> EIGRP	拒绝 Tag `100`	`set tag 1`	OSPF 原生路由进入 EIGRP
EIGRP -> OSPF	拒绝 Tag `1`	`set tag 100`	EIGRP 原生路由进入 OSPF

逻辑：

```
OSPF 路由进 EIGRP：set tag 1
带 tag 1 的路由不允许再进 OSPF

EIGRP 路由进 OSPF：set tag 100
带 tag 100 的路由不允许再进 EIGRP
```

17.5 R3/R4：配置双向重分发

下面这组策略在 R3、R4 两台 ASBR 上都配置。

先定义允许重分发的 Loopback0 前缀：

```
ip prefix-list OSPF-TO-EIGRP-PFX seq 10 permit 1.1.1.1/32
ip prefix-list OSPF-TO-EIGRP-PFX seq 20 permit 2.2.2.2/32
ip prefix-list OSPF-TO-EIGRP-PFX seq 30 permit 3.3.3.3/32
ip prefix-list OSPF-TO-EIGRP-PFX seq 40 permit 4.4.4.4/32

ip prefix-list EIGRP-TO-OSPF-PFX seq 10 permit 5.5.5.5/32
ip prefix-list EIGRP-TO-OSPF-PFX seq 20 permit 6.6.6.6/32
```

OSPF 注入 EIGRP，打 Tag `1`：

```
route-map OSPF-TO-EIGRP deny 10
 match tag 100

route-map OSPF-TO-EIGRP permit 20
 match ip address prefix-list OSPF-TO-EIGRP-PFX
 set metric 100000 100 255 1 1500
 set tag 1

router eigrp 100
 redistribute ospf 1 route-map OSPF-TO-EIGRP
```

EIGRP 注入 OSPF，打 Tag `100`：

```
route-map EIGRP-TO-OSPF deny 10
 match tag 1

route-map EIGRP-TO-OSPF permit 20
 match ip address prefix-list EIGRP-TO-OSPF-PFX
 set metric 20
 set metric-type type-1
 set tag 100

router ospf 1
 redistribute eigrp 100 subnets route-map EIGRP-TO-OSPF
```

解释：

- Prefix-list 负责限制只重分发 Loopback0，不把两边的互联链路全部带过去。
- `OSPF-TO-EIGRP deny 10` 匹配 Tag `100`，挡住 EIGRP 原生路由重新回到 EIGRP。
- `EIGRP-TO-OSPF deny 10` 匹配 Tag `1`，挡住 OSPF 原生路由重新回到 OSPF。
- R3、R4 都是 ASBR，所以两台设备都要配置同一套重分发策略。

17.6 验证

OSPF 左侧验证：

```
R1# show ip route 5.5.5.5
R1# show ip route 6.6.6.6
R1# ping 5.5.5.5
R1# ping 6.6.6.6
```

预期：R1 能看到来自 EIGRP 的 OSPF 外部路由，并能访问 R5/R6 的 Loopback0。

EIGRP 右侧验证：

```
R5# show ip route 1.1.1.1
R5# show ip route 2.2.2.2
R5# show ip route 3.3.3.3
R5# show ip route 4.4.4.4
R5# ping 1.1.1.1
R5# ping 2.2.2.2
R5# ping 3.3.3.3
R5# ping 4.4.4.4

R6# show ip route 1.1.1.1
R6# show ip route 2.2.2.2
R6# show ip route 3.3.3.3
R6# show ip route 4.4.4.4
```

预期：R5/R6 能看到来自 OSPF 的 EIGRP External 路由，并能访问 R1/R2/R3/R4 的 Loopback0。

ASBR 验证：

```
R3# show route-map
R3# show ip prefix-list
R3# show ip ospf database external
R3# show ip eigrp topology

R4# show route-map
R4# show ip prefix-list
R4# show ip ospf database external
R4# show ip eigrp topology
```

重点看：

- Route-map 的 permit 条目是否有命中。
- Route-map 的 deny 条目是否在回流场景中有命中。
- OSPF External LSA 是否带 Tag `100`。
- EIGRP External 路由是否带 Tag `1`。

17.7 排错顺序

如果实验不通，按这个顺序查：

1. 查邻居：`show ip ospf neighbor`、`show ip eigrp neighbors`。
2. 查边界 RIB：`show ip route ospf`、`show ip route eigrp`。
3. 查 Prefix-list 是否允许目标前缀。
4. 查 Route-map 是否命中。
5. 查 `redistribute` 是否已经在协议下调用。
6. 查 Tag 是否存在，回流方向是否被 deny。
7. 查最终路由下一跳是否正确。

记忆顺序：

```
先有邻居 -> 再有来源路由 -> 再匹配策略 -> 再重分发 -> 最后查 Tag 和下一跳
```

18. 查看命令

Prefix-list：

```
show ip prefix-list
show ip prefix-list NAME
show ip prefix-list detail NAME
```

ACL：

```
show access-lists
show ip access-lists
```

Route-map：

```
show route-map
```

路由协议：

```
show ip protocols
show ip route
show ip route 10.10.10.0
show ip cef 10.10.10.10 detail
```

OSPF：

```
show ip ospf database
show ip ospf database summary
show ip ospf database external
show ip route ospf
```

EIGRP：

```
show ip eigrp topology
show ip route eigrp
```

19. 常见故障

现象	常见原因	排查方向
路由没有被重分发	路由不在 RIB、Route-map 未匹配、缺少 Seed Metric	查 `show ip route` 、 `show route-map` 、 `show ip protocols`
重分发进 OSPF 后缺少子网	OSPF `redistribute` 未加 `subnets`	查 `router ospf` 配置
路由被过滤过多	Prefix-list 只有 deny，没有后续 permit	查 Prefix-list 命中和隐式 deny
OSPF 过滤后 LSDB 仍有 LSA	OSPF 入向 distribute-list 只影响本地 RIB	对比 `show ip ospf database` 和 `show ip route`
OSPF 区域间过滤不生效	Filter-list 方向写反、没有配置在 ABR、试图过滤同区域内部 LSA	查 `area filter-list prefix` 、Prefix-list 命中和 `show ip ospf database summary`
多点重分发路径异常	未设置 Tag，Metric 不一致，AD 选择异常	查 Tag、AD、Metric、边界路由器
路由反馈	双向重分发无 Tag 过滤	查路由来源和 Tag
外部路由优于内部路由	AD 被修改或外部路由类型变化	查路由来源和 AD
边缘明细过多进入核心	缺少 Prefix-list 或 Route-map 限定	查重分发策略

20. 小结

- 路由控制决定路由信息是否被学习、通告、重分发和安装。
- Prefix-list 比 ACL 更适合路由前缀过滤，因为它能精确匹配掩码长度。
- Distribute-list 在 EIGRP 与 OSPF 中效果不同；OSPF 不能随意破坏同区域 LSDB 一致性，区域间过滤要使用 Filter-list。
- Route-map 是路由策略核心工具，permit/deny 的含义取决于调用场景。
- 重分发只会把本机 RIB 中的路由注入目标协议。
- 重分发必须设计 Seed Metric，否则不同协议的 Metric 无法直接比较。
- 单点单向最安全，多点双向最危险。
- 重分发导致的问题要结合拓扑看：AD、Metric、边界位置共同决定是否次优或环路。
- 多点或双向重分发必须使用 Prefix-list、Route-map、Tag、Metric、AD 和默认路由进行控制。